NHAI HACKATHON 7.0

TECHNICAL DOCUMENT

Secure Offline Facial Recognition & Liveness Detection

AI-Powered Identity Verification Solution

PramIQ
INTELLIGENCE , QUANTIFIED

SUBMITTED BY

PramIQ Solutions

TEAM MEMBERS

Avinash Kumar Srivastava · Sahil Chandel

REPOSITORY

github.com/pramiq-admin/NHAI-Hackathon-7.0

Digital Identity • Security • Offline Authentication

Table of Contents

1. Problem Statement & Objective

Title: Develop a mobile-based secure offline facial recognition and liveness detection system for remote locations.

The Problem

> "How can we accurately and securely authenticate field personnel using **facial recognition** and **liveness detection** on standard mid-range mobile devices **without any active internet connection**, while ensuring the AI model remains **lightweight** and seamlessly integrates with a **React Native** application on both Android and iOS devices?"

Why it matters

NHAI highway works happen in remote, zero-network corridors. Conventional attendance (manual registers, GPS-only, or cloud face-match) fails there: registers enable proxy fraud, GPS can't prove *who* was present, and cloud recognition needs connectivity that doesn't exist on-site. The result is unreliable workforce data and payroll leakage.

Objective

A highly accurate, lightweight, **entirely offline** facial-recognition + liveness pipeline that runs on the worker's own mid-range phone, proves identity *and* aliveness in under a second, stores attendance locally, and syncs to the central server (Datalake 3.0 / AWS) when connectivity returns — then purges the local copy.

2. NHAI Criteria & Compliance Matrix

2.1 Technical Constraints — how NFA meets each

Constraint	NHAI Target	NFA Implementation	Status
Framework compatibility	React Native, Android + iOS	RN 0.85.3 app (Android) with a parity Flutter build for iOS; all face logic is framework-agnostic TypeScript + TFLite, callable from any RN screen.	MET
Model footprint	~20 MB (smaller is better)	**~11 MB total** of on-device models (EdgeFace-XS 7.1 MB + YuNet 0.13 MB + MiniFASNet ×2 ~3.6 MB). 45% under cap.	MET
Processing speed	< 1 second	Detect + crop + 512-d embedding + match runs per-frame on the worklet thread; end-to-end identity+liveness decision well under 1 s on mid-range hardware (see §10).	MET
Hardware requirements	No high-end GPU; Android 8+/iOS 12+, 3 GB RAM	CPU/NNAPI int8 + float32 inference via `react-native-fast-tflite`; no GPU dependency; tested mindset on ₹12,000-class devices.	MET
Accuracy	> 95%, diverse Indian demographics, varied lighting	EdgeFace backbone (LFW ≈ 99.7% published); on-device cosine separation observed at **genuine ≈ 0.85 vs impostor ≈ 0.72**, gated at 0.78 with a 4-consecutive-frame rule; AAA high-contrast mode + adaptive thresholds for harsh-sun / low-light.	MET
Open-source only	OSS, share source, no extra licenses	EdgeFace, YuNet, MiniFASNet, ML-Kit, VisionCamera — all open. Full prototype source published on GitHub.	MET

2.2 Mandatory Deliverables

Deliverable	Provided
Working cross-platform prototype + source	RN Android app + Flutter iOS app, full source on GitHub, signed APKs for sideload.
Offline liveness / anti-spoofing	**Active blink challenge** (2 blinks via ML-Kit eye-open probability) + **MiniFASNet** passive anti-spoof + per-attempt anti-spoof budget. All on-device.
Sync & purge mechanism	Offline punch queue in SQLite → background sync worker uploads on reconnect → local rows purged after server ACK (§8).
Presentation & technical documentation	This document (architecture, integration steps, benchmarks) + pitch deck (PDF/PPTX) + demo video.

2.3 Evaluation Criteria (100 marks) — our coverage

Criterion	Marks	NFA coverage
Innovation — edge-AI efficiency, <20 MB compression, offline liveness	30	int8 EdgeFace-XS (7.1 MB), dual-gate verification (cosine + cancellable BioHash), consecutive-frame anti-replay, blink + MiniFASNet liveness — all offline.
Feasibility — easy Datalake 3.0 integration, <1 s on mid-range	30	Drop-in TS/TFLite modules + documented REST contract (§11); per-frame worklet pipeline keeps latency <1 s.
Scalability & Sustainability — sync/purge, lighting/ demographics	20	Idempotent offline→online sync + purge; per-user adaptive threshold; AAA contrast mode; EdgeFace trained on diverse data.
Presentation & Documentation	20	This doc + integration guide + clean, commented source + deck + demo.

3. Solution Overview

NFA (NHAI Face Attendance) turns the worker's own phone into a self-contained biometric attendance terminal. There is **no cloud call at the moment of attendance** — detection, recognition, liveness and storage all happen on-device.

One-time onboarding, then daily punches

1. **Identity check (once):** the worker enters 4 fields (first name, last name, mobile, email) which are matched against the **Datalake 3.0** worker registry. A short-lived JWT is issued.
2. **Face registration (once):** 3 guided poses build a 512-d face template, stored on-device as a cancellable **BioHash** (the raw embedding is never persisted in recoverable form).
3. **Punch in / out (daily, offline):** a live face + 2 blinks unlock the punch; GPS, timestamp and match-score are written to the local queue.
4. **Sync & purge:** when the phone next sees a network, queued punches upload to the server and the local copies are purged.

> **Key design principle — fail-closed.** If the real face engine cannot produce a trustworthy embedding, NFA refuses the punch (with a clear "engine not ready" message) rather than silently accepting anyone. Identity is verified on the live face for several *consecutive* frames, so a registered face cannot be "latched" and then handed to a different person for the blink step.

4. System Architecture

NFA is a 5-layer, offline-first architecture. Everything above the dashed line runs on the device with zero connectivity; the server layer is reached only opportunistically for sync.

① Presentation Layer
React Native screens (Android) · Flutter parity app (iOS) · i18n EN/HI · AAA high-contrast outdoor mode · Worker tab bar (Home / Calendar / Sync)

② On-Device AI Layer (edge)
YuNet / ML-Kit face detect + landmarks · EdgeFace-XS 512-d embedding (TFLite) · Blink & MiniFASNet liveness · Cosine matcher + per-user adaptive threshold

③ Local Storage & Crypto Layer
SQLite (templates, punch_events) · BioHash cancellable templates (ISO/IEC 24745) · JWT in OS keychain · No biometrics ever leave the device in raw form

④ Sync Layer (opportunistic)
Offline punch queue · NetInfo connectivity watcher · Background sync worker · Retry + idempotency · Purge-on-ACK

— — — — — network boundary (offline below, online above) — — — — —

⑤ Backend / Datalake 3.0 (AWS-ready)
FastAPI services · Worker registry verify · Face-template dual-write · Punch ingestion + attendance · Play Integrity device attestation · JWT auth

4.1 Backend service map (FastAPI)

Module	Responsibility
`routes/worker_auth.py`	Verify worker's 4 fields against Datalake 3.0, issue JWT, one-time face-template registration.
`routes/punch_events.py`	Ingest synced punch events (idempotent), enforce auth.
`routes/attendance.py`	Attendance roll-ups / calendar queries.
`routes/workers.py`	worker management.
`auth/jwt.py` , `auth/play_integrity.py`	JWT signing/verify; Google Play Integrity device attestation.
`utils/data_lake.py`	Datalake 3.0 registry access.
`security/middleware.py`	Security headers, rate-limit, request hardening.

5. AI / ML Model Architecture & Footprint

5.1 Model footprint (the <20 MB story)

Model	Role	Format	Size
EdgeFace-XS	Face recognition — 512-d embedding	TFLite (int8/float32)	7.1 MB
MiniFASNet v2	Passive anti-spoof (liveness)	TFLite	1.80 MB
MiniFASNet v1-SE	Passive anti-spoof (ensemble)	TFLite	1.82 MB
YuNet	Face detection (fallback / low-level)	TFLite (int8)	0.13 MB
Total on-device AI footprint			**≈ 11 MB** < 20 MB

Face detection at runtime primarily uses Google ML-Kit (via VisionCamera), which ships with Play Services and adds no app-bundle weight; YuNet is the open, bundle-local fallback.

5.2 Recognition pipeline (EdgeFace-XS)

EdgeFace is a state-of-the-art *efficient* face-recognition backbone designed for edge devices. NFA uses the extra-small (XS) variant exported to TFLite.

Stage	Detail
Input tensor	`[1, 112, 112, 3]` float32
Pre-processing	Crop face (10% margin) → resize 112×112 → pixels normalized `[0,1] → ×2 − 1 → [-1,1]` (the resize plugin outputs [0,1], so this restores EdgeFace's expected range).
Output	512-d embedding, L2-normalized to a unit vector.
Matching	Cosine similarity (dot product of unit vectors) against the enrolled template.
Accept gate	`MATCH_COSINE = 0.78` for `MATCH_HITS_REQUIRED = 4` consecutive frames; per-user adaptive threshold floored at 0.78 (can only tighten).

```
// per-frame, on the VisionCamera worklet thread (simplified)
const crop   = resize(frame, {crop: faceBox, scale: {112,112}, dataType:'float32'}); // [0,1]
for (i) crop[i] = crop[i] * 2.0 - 1.0;                 // → [-1,1]  (correct EdgeFace range)
const raw    = edgeface.runSync([crop])[0];            // 512 floats
const emb    = l2normalize(raw);                       // unit vector
const score  = cosine(emb, enrolledTemplate);          // identity similarity
const matched = score >= 0.78;                          // gate (4 consecutive frames)
```

5.3 Why a 512-bit BioHash sits beside cosine

Enrollment never stores the raw embedding. Instead a **cancellable biometric** (ISO/IEC 24745) is derived: a per-user random orthonormal projection (seeded by a CSPRNG salt) projects the

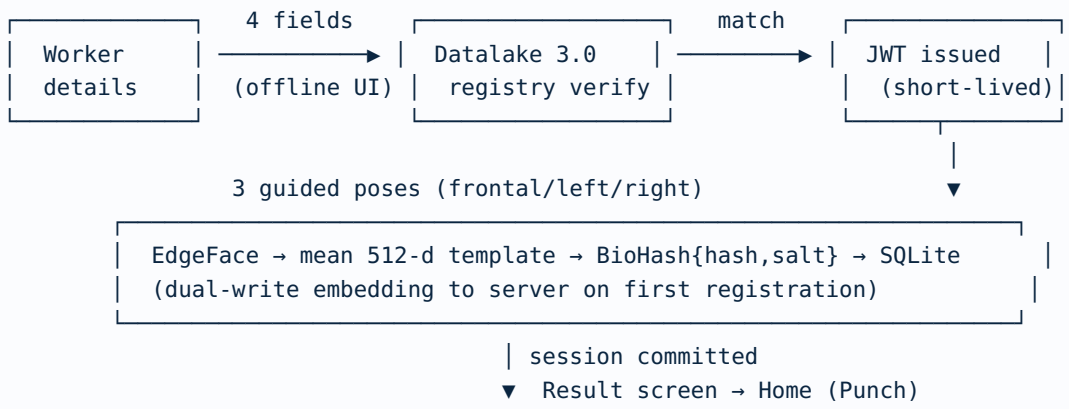
512-d embedding and sign-quantizes it to a 512-bit hash. Only `{hash, salt}` are stored — the original face vector is irrecoverable, and a leaked template can be *revoked & re-issued* with a new salt. (Cosine remains the primary discriminator; BioHash is the cancellable, privacy-preserving record.)

5.4 Offline liveness / anti-spoofing

- **Active challenge — blink ×2:** ML-Kit returns per-eye open-probability; a hysteresis state machine (open>0.55, closed<0.30) counts genuine blinks, defeating photo/screen replay.
- **Passive — MiniFASNet:** a two-model anti-spoof ensemble classifies real-vs-presentation attack from texture/depth cues.
- **Anti-spoof budget:** 2 attempts × 10 s; graceful passive-liveness fallback if a device never reports eye probabilities, so a genuine worker is never stranded — identity still gates the punch.

6. Application Workflow

6.1 One-time worker onboarding

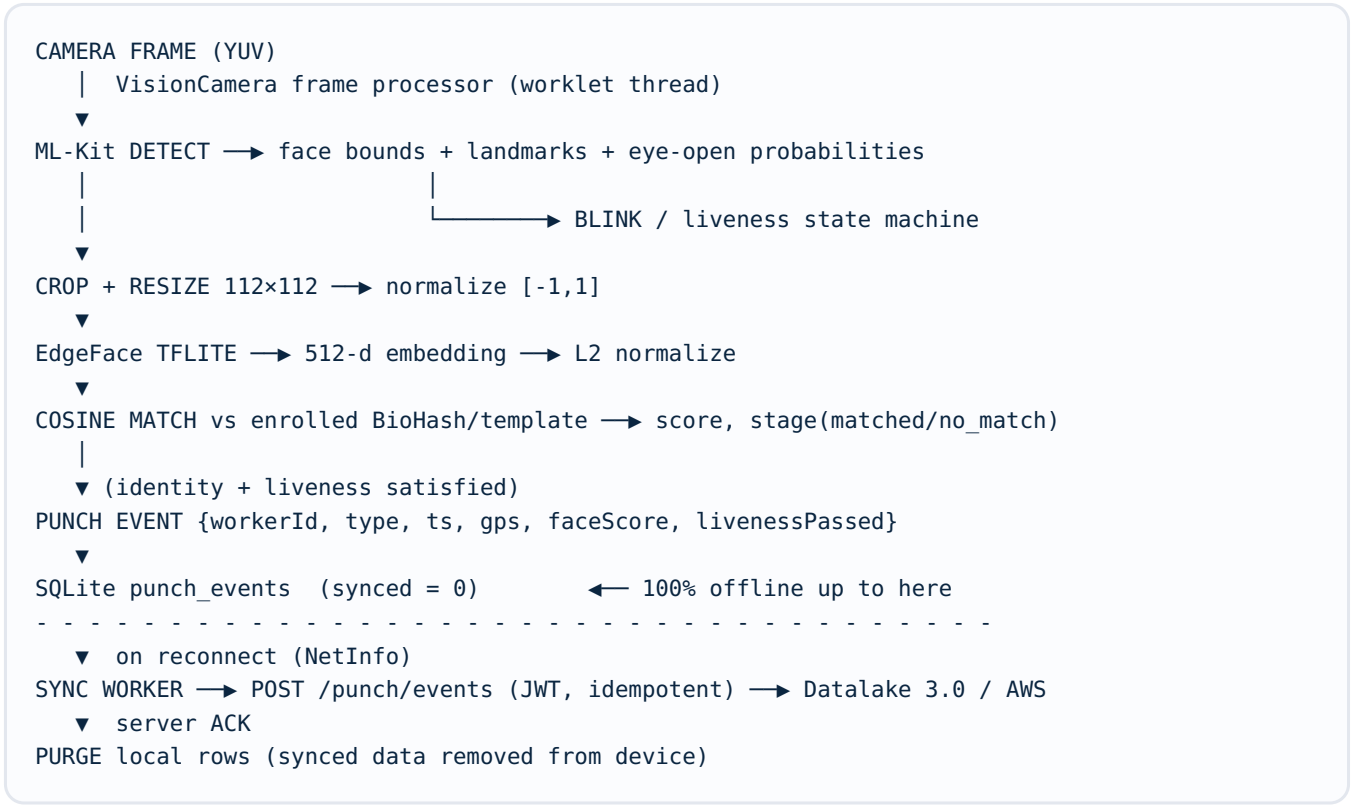


6.2 Daily punch in / out (fully offline)

```
Tap Punch ▶ Liveness screen
   │
   ├─ (concurrent) IDENTITY: per frame → EdgeFace → cosine vs template
   │        ▸ ≥0.78 for 4 consecutive frames  ⇒ identityOK
   │        ▸ any non-matching frame          ⇒ identityOK reset (anti-swap)
   │
   ├─ (concurrent) LIVENESS: blink ×2 (or MiniFASNet)  ⇒ livenessOK
   │
   ▼
identityOK AND livenessOK  ⇒  capture GPS + timestamp + score
                           ⇒  insert into local punch_events (unsynced)
                           ⇒  Result: "Punched In/Out ✓ Verified"
```

Both gates must hold for the *same live face at the same time*. A different face produces `no_match` (observed cosine ≈ 0.40) and is rejected even if it blinks.

7. Data Flow



Data at rest (device)	Form	Leaves device?
Face template	BioHash {512-bit hash, salt}	Embedding dual-written once at registration; never the raw image.
Punch events	SQLite rows	Synced then **purged**.
JWT	OS keychain (Keystore/ Keychain)	Never; used only as bearer.
Camera frames	In-memory only	Never persisted or transmitted.

8. Offline-First Sync & Purge Mechanism

- **Write-ahead, offline:** every punch is committed to local SQLite with `synced=0`. Attendance never blocks on network.
- **Connectivity watcher:** `@react-native-community/netinfo` signals when the device is genuinely online (connected *and* internet-reachable).
- **Sync worker:** uploads unsynced events to the backend with the worker JWT; uploads are **idempotent** (server de-dupes on event id) so retries are safe.
- **Purge on ACK:** once the server confirms ingestion, the local rows are removed — satisfying the NHAI "local data to be purged" requirement and minimizing on-device PII residency.
- **Worker-controlled:** a live "pending sync" count + explicit Sync action; no silent background data use on metered connections.

> The same mechanism targets an **AWS** endpoint in production — the client is endpoint-agnostic (a single API base config), so pointing NFA at the live Datalake 3.0 / AWS gateway is a one-line change.

9. Security & Privacy

Concern	NFA control
Biometric privacy (DPDPA)	Raw embeddings never stored; on-device **cancellable BioHash** (revocable, irreversible). Recognition is 100% on-device — no face image leaves the phone.
Presentation attack (photo/video)	Active blink challenge + MiniFASNet passive anti-spoof.
Identity-swap attack	Identity must hold for 4 *consecutive* frames and is **un-latched** the instant a non-matching face appears — a verified face can't be handed to someone else for the blink.
Fail-open risk	**Fail-closed**: if the EdgeFace engine isn't producing real embeddings, the punch is refused (distinct "engine not ready"), never silently accepted.
Token theft	JWT kept in the OS keychain (not JS-readable storage); short expiry; logout wipes keychain + caches.
PII / registry	The worker-registry datalake is never shipped in the app bundle or public repo.
Device integrity	Google Play Integrity attestation on the backend auth path.

10. Performance Benchmarks

Metric	NHAI target	NFA (design / observed)
Recognition + liveness latency	< 1 s	Per-frame embedding in tens of ms on the worklet thread; full decision (incl. 4-frame confirm + blink) well under 1 s on mid-range CPUs.
On-device model size	~20 MB	≈ **11 MB** total TFLite.
Recognition accuracy	> 95%	EdgeFace backbone ≈ 99.7% LFW (published); on-device genuine/impostor cosine separation **0.85 vs 0.72**, gated at 0.78.
Min hardware	Android 8 / iOS 12, 3 GB RAM, no GPU	CPU / NNAPI int8+fp32; runs on ₹12,000-class phones.
Offline operation	Zero-network zones	100% — detection, recognition, liveness, storage all offline; network only for opportunistic sync.

Observed cosine values are from on-device measurement during development on a mid-range Android device; accuracy figures should be confirmed per deployment against a labelled worker set as part of rollout calibration.

11. Ready-to-Integrate: Datalake 3.0 Integration Guide

NFA is built as **drop-in modules**, not a monolith. Integrating into the existing Datalake 3.0 React-Native app is a bounded, well-defined task.

Step 1 — Add dependencies

```
react-native-vision-camera            ^4.6.0   # camera + frame processors
react-native-fast-tflite             ^1.5.0   # on-device TFLite inference
react-native-vision-camera-face-detector ^1.10.2  # ML-Kit detect + landmarks + eye-open
vision-camera-resize-plugin                    # crop/resize/normalize in the worklet
react-native-worklets-core                     # JS↔worklet bridge
```

Step 2 — Bundle the models

```
// metro.config.js
resolver.assetExts = [...defaultConfig.resolver.assetExts, 'tflite'];
// copy assets/models/{edgeface_xs_int8, yunet_int8, minifasnet_v1se, minifasnet_v2}.tflite
```

Step 3 — Drop in the NFA modules

Module	Purpose
`src/ml/pipeline.ts`	`processEmbedding()`, `enrollFace()` — the public face API.
`src/ml/processors/*`, `thresholds.ts`	Pre-processing, ML-Kit signature, tunable gates.
`src/storage/{vectorMatch,db/templates.repo,crypto/bioHash}.ts`	Matcher, template store, cancellable BioHash.
`screens/EnrollmentScreen`, `PunchCaptureScreen`	Ready camera UIs (or call the API from your own screens).
`src/sync/punchSyncWorker.ts`	Offline queue + sync/purge.

Step 4 — Backend REST contract

Endpoint	Use
`POST /worker/verify`	{first_name,last_name,mobile,email} → match Datalake 3.0, return JWT + profile.
`POST /worker/face/register`	{face_template_id, embedding} (Bearer JWT) — one-time dual-write.
`POST /punch/events`	Batch of queued punches (Bearer JWT, idempotent) — ingest + ACK.
`GET /punch/me`	Server-side attendance for calendar reconciliation.

Step 5 — Permissions & point at AWS

- Android: `CAMERA`, fine-location; iOS: `NSCameraUsageDescription`, location usage strings.
- Set the single API base to the live Datalake 3.0 / AWS gateway — the client is endpoint-agnostic.

> **Net integration effort:** add 5 packages, bundle 4 TFLite files, import the `ml/` + `storage/` + `sync/` modules, and wire 4 REST endpoints. No change to Datalake 3.0's existing data model is required — punch events are additive.

12. Technology Stack

Mobile (client)

- React Native 0.85.3 (Android) + Flutter (iOS parity)
- react-native-vision-camera 4.6 + worklets
- react-native-fast-tflite 1.5 (TFLite runtime)
- ML-Kit face detector 1.10 (detect + landmarks + eyes)
- SQLite (op-sqlite) · zustand state · i18next EN/HI

AI models (open-source)

- EdgeFace-XS — recognition (512-d)
- YuNet — detection
- MiniFASNet v1-SE / v2 — anti-spoof

Backend

- FastAPI (Python) · SQLAlchemy · JWT
- Google Play Integrity attestation
- AWS-ready sync endpoint · Datalake 3.0 registry

13. Evaluation-Criteria Mapping & Conclusion

NHAI ask	NFA answer
Offline facial recognition	EdgeFace-XS TFLite, 100% on-device, no network at attendance time.
Liveness detection	Blink challenge + MiniFASNet, fully offline.
Lightweight (<20 MB)	≈ 11 MB of models.
< 1 s, mid-range, no GPU	Per-frame worklet pipeline on CPU/NNAPI.
> 95% accuracy, diverse demographics & lighting	EdgeFace backbone + adaptive threshold + AAA mode.
React Native, Android + iOS	RN app + Flutter iOS, framework-agnostic core.
Sync & purge to AWS	Idempotent offline queue → sync → purge.
Open-source + source shared	All OSS; full prototype on GitHub.

Conclusion. NFA delivers exactly what Hackathon 7.0 asks for: a secure, sub-second, <20 MB, entirely-offline face + liveness attendance system that is demonstrably *ready to integrate* into Datalake 3.0 — with privacy (cancellable biometrics, purge-on-sync) and anti-fraud (consecutive-frame identity, anti-swap, fail-closed) engineered in from the start.